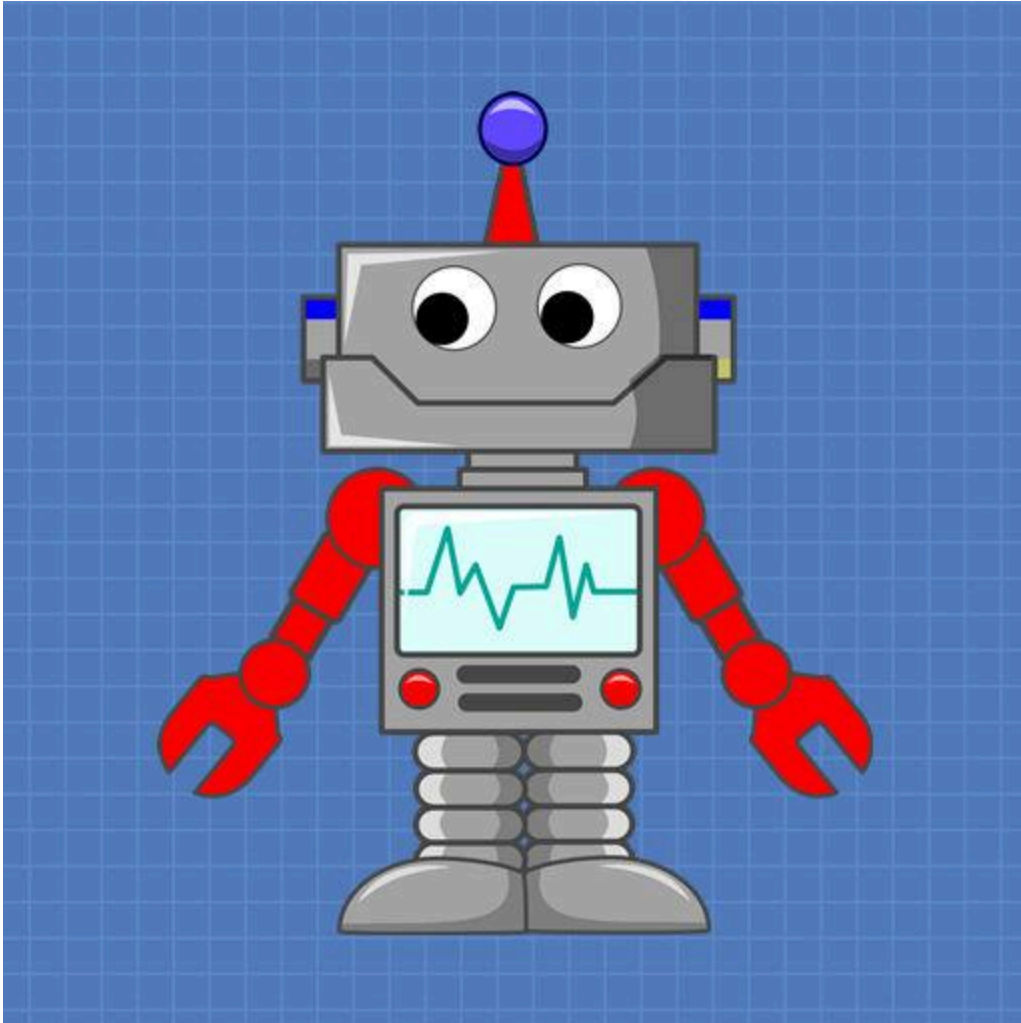


ESP32 Bluetooth



DroneBot Workshop Tutorial

<https://dronebotworkshop.com>

For more projects and tutorials visit the DroneBot Workshop - <https://dronebotworkshop.com>

Today, we will learn how to use the ESP32's Bluetooth and BLE features to communicate with phones, tablets, sensors, and other ESP32 boards.



Introduction

In 1994, engineers Nils Rydbeck, Johan Ullman, and Dr. Jaap Haartsen at the Swedish technology company Ericsson conceived a method to replace the ever-expanding jungle of computer cables with a wireless system. Four years later, Ericsson partnered with IBM, Intel, Nokia, and Toshiba to form the [Bluetooth Special Interest Group \(SIG\)](#). One year later, in 1999, the specifications for Bluetooth 1.0 were released.



Bluetooth (named after the 10th-century king Harald “Bluetooth” Gormsson, who united Denmark and Norway) was initially designed to replace RS-232 (serial) cables with a 2.4GHz wireless link. It has since been expanded to include audio streaming and direction finding.

In 2010, Bluetooth Low Energy, or BLE, was introduced as part of the Bluetooth 4.0 specifications. This low-powered technology was designed for short data bursts like sensor data. BLE devices have very low power consumption, and the technology is used in items like smartwatches and fitness trackers.

The Espressif ESP32 microcontroller is an ideal component for hobbyists who want to build Bluetooth and BLE projects. It’s inexpensive, easy to use, and well-documented.

Today, we will see how easy it is to get started with Bluetooth on the ESP32.

Bluetooth Classic & BLE

Both Bluetooth (now called *Bluetooth Classic*) and BLE (formerly known as *Bluetooth Smart*) use the 2.4 GHz ISM (Industrial—Scientific—Medical) band, which is available worldwide for non-licensed low-powered radio communications.

Bluetooth (both flavors) is designed to communicate over short distances, making it ideal for connecting computers and mobile devices to peripherals.

Similarities and Differences

The two protocols are used for different purposes:

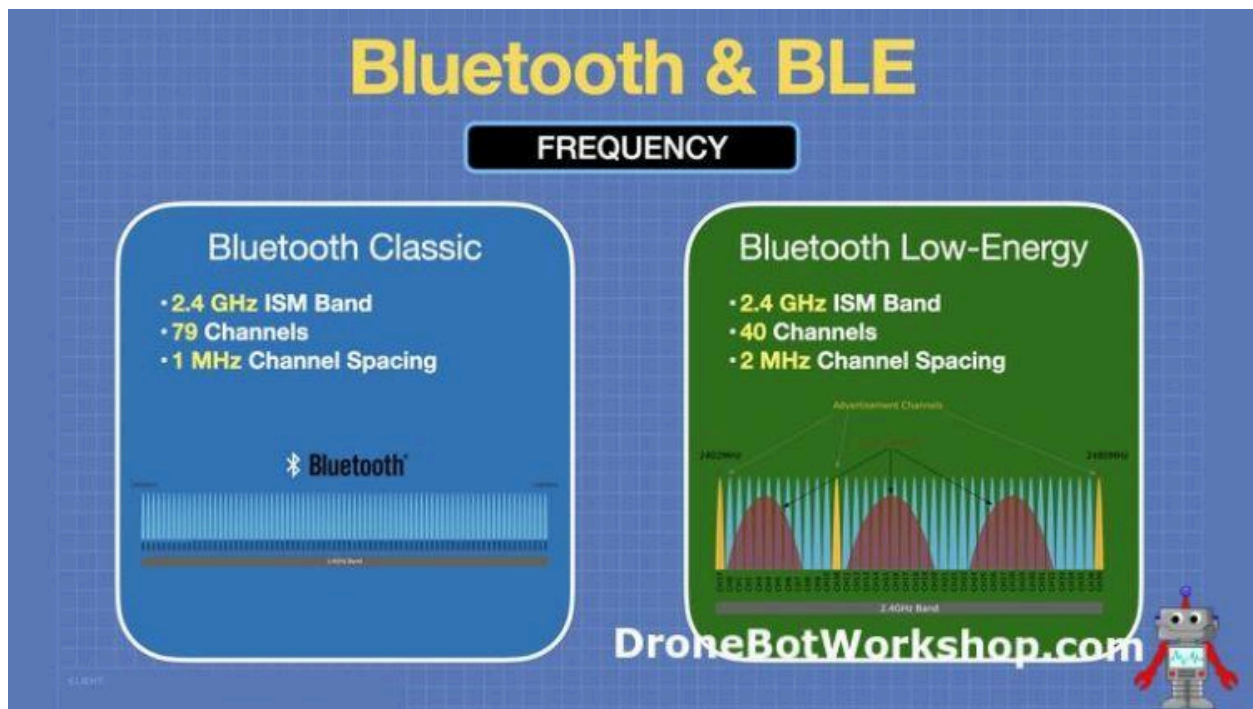
- **Bluetooth Classic (BR/EDR)** – The original Bluetooth, designed for streaming data like audio. It prioritizes speed and continuous connections.
- **Bluetooth Low Energy (BLE)** – Focused on low power consumption and periodic data transfers, perfect for battery-powered devices and sensors.

Both protocols have their roots in the original Bluetooth 1.0 standard. They share several characteristics but also have differences. They can also be used concurrently, for example, an audio device may use Bluetooth Classic for audio transmission while also using BLE for control signals.

Frequency

Both Bluetooth Classic and BLE operate in the 2.4GHz ISM band, which extends from 2.400 GHz to 2.4835 GHz. However, the two standards differ in how they divide the bandwidth and are thus incompatible with one another.

- Bluetooth Classic divides the band into 79 channels, with a spacing of 1 MHz.
- BLE divides the band into 40 channels, with a spacing of 2 MHz.

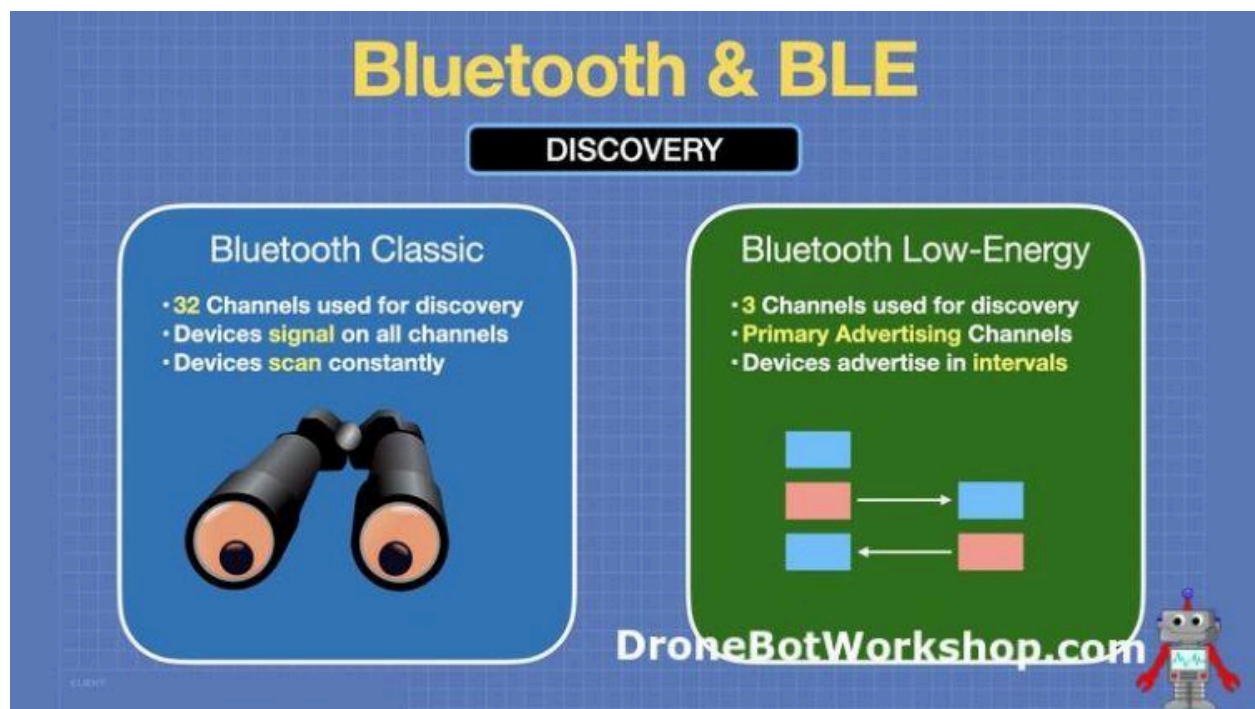


Both protocols are limited in distance. Bluetooth Classic is rated to exchange data up to 100 meters, while BLE has half that range. However, in real life, distances are usually much shorter, especially indoors.

Device Discovery

Device Discovery is the process by which Bluetooth devices make their presence known to one another.

- Each device scans for other devices.
- When a device wants to join, it sends a signal.
- The joining process can then take place. This can take several seconds.



The concept is pretty basic, but each protocol does it differently.

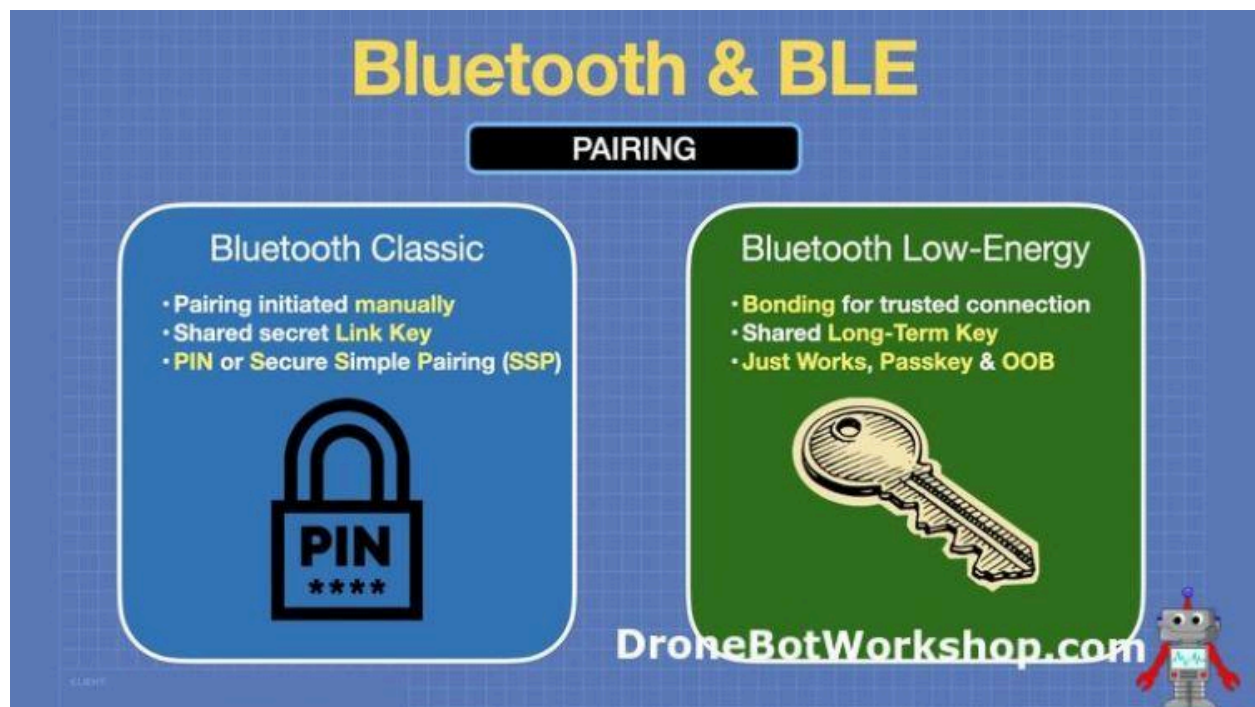
In Bluetooth Classic, 32 of the 79 channels can be used to establish a connection between devices. When a device wants to be discovered, it sends signals on these channels.

For more projects and tutorials visit the DroneBot Workshop - <https://dronebotworkshop.com>

BLE only has three channels that can be used to establish a connection. A BLE device that wants to be discovered sends signals on channels 37, 38, and 39, which are known as the *Primary Advertising Channels*.

Device Pairing

Bluetooth Classic and BLE *Device Pairing* is a secure process that enables two devices to communicate by establishing a trusted connection. Once again, both protocols do it differently.



In Bluetooth Classic, pairing is initiated manually by the user. The devices then exchange a shared secret, known as a *link key*. Once the link key is exchanged, the devices are paired and can establish a secure connection whenever they're in range.

Some devices may require a PIN for pairing, while others use Secure Simple Pairing (SSP), simplifying the process and improving security.

<https://dronebotworkshop.com>

For more projects and tutorials visit the DroneBot Workshop - <https://dronebotworkshop.com>

BLE uses a process called “bonding” to create a trusted connection between devices. To lower power consumption, the devices exchange and store a *long-term key*. This key is used to encrypt future connections, ensuring that only bonded devices can communicate with each other.

BLE supports three pairing methods: Just Works, Passkey Entry, and Out of Band (OOB). The method used depends on the capabilities of the devices involved.

Data Exchange

In classic Bluetooth, data exchange occurs over a stable, continuous connection between two paired devices. It creates channels called profiles, which determine how devices interact. Examples are sending music (A2DP profile) or controlling hands-free devices (HFP profile). These connections are maintained for extended data flow, such as streaming.

Bluetooth & BLE

DATA EXCHANGE

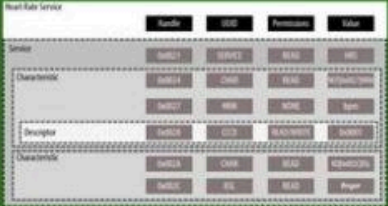
Bluetooth Classic

- **Continuous** connection
- **Profiles** determine interaction
- Allows for **Streaming**



Bluetooth Low-Energy

- **Intermittent** connection
- **Generic Attribute Profile (GATT)**
- **Services & Characteristics**



DroneBotWorkshop.com



For more projects and tutorials visit the DroneBot Workshop - <https://dronebotworkshop.com>

BLE is designed for low-power, intermittent data transfer. Data is exchanged using the *Generic Attribute Profile (GATT)*.

GATT operates using *Services* and *Characteristics*. A Service is a collection of data and associated behaviors to accomplish a particular function or feature of a device.

Characteristics are the individual pieces of data that can be read, written to, or notified of changes by the device.

GATT is like an apartment building mailbox with compartments: *Services* are the mailboxes themselves, and *Characteristics* represent the mail in the box. You can read, write, or receive notifications when data in a Characteristic changes, perfect for sensors or remote controls sending occasional data.

We will examine GATT in more detail when we work with BLE.

ESP32 with Bluetooth Classic & BLE

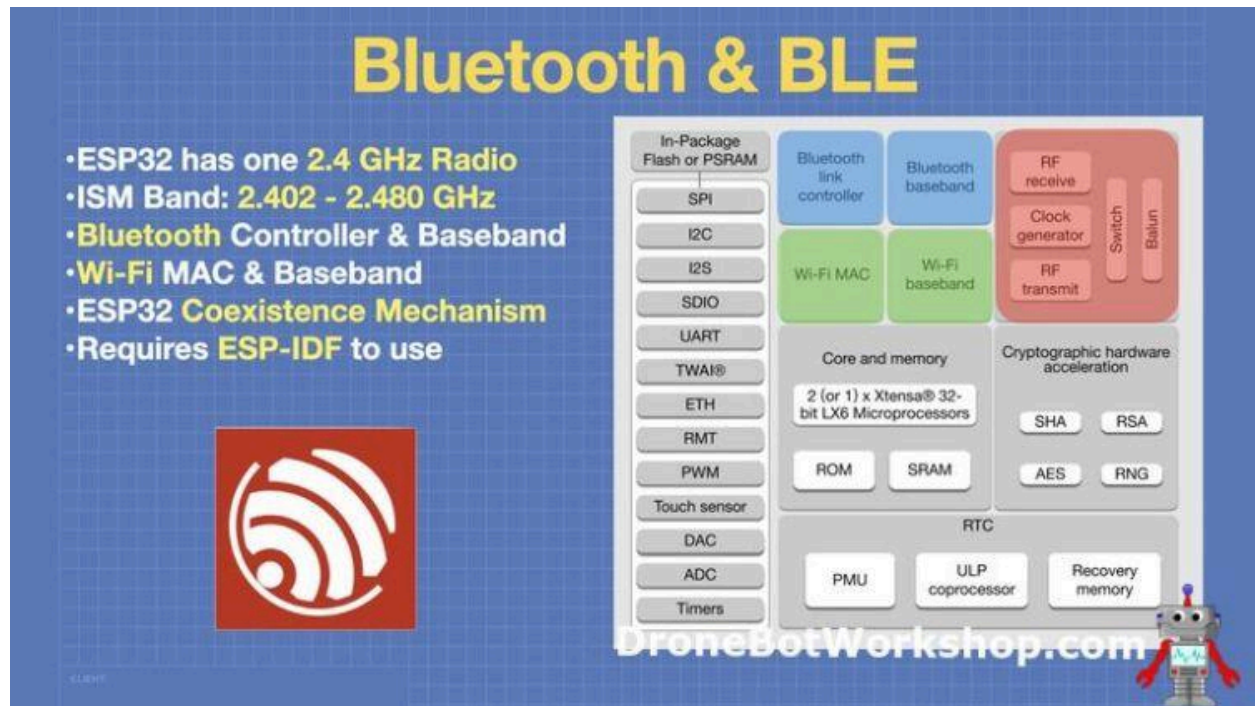
The ESP32 is an excellent choice for working with both Bluetooth Classic and BLE. It supports both protocols, and Espressif provides libraries and code samples to get you up and working quickly.

There are many models of the ESP32; most of them support Bluetooth, but a few do not. Check out our 2024 ESP32 Guide to find a Bluetooth-capable processor that suits your requirements.

The ESP32 has a wealth of I/O ports, both digital and analog. It's available in several configurations from Espressif and other manufacturers.

ESP32 2.4GHz Radio

The ESP32 microcontroller features a versatile 2.4 GHz radio section that is capable of handling both Wi-Fi and Bluetooth communication. This radio operates in the license-free 2.4 GHz ISM band.



As there is only one radio module, using Bluetooth and Wi-Fi simultaneously might not seem possible, but it can be done. The ESP32 has a built-in “coexistence mechanism” to manage the shared radio resource. This involves intelligent scheduling and time-slicing to give both Wi-Fi and Bluetooth a chance to transmit and receive data.

This is a pretty advanced technique, and you must use the Espressif ESP-IDF development platform to implement it. Even when implemented properly it is still subject to errors during periods of high data throughput.

You can read more about [RF Coexistence on the Espressif website](#).

Experimenting with Bluetooth & ESP32

Now that we know a bit more about Bluetooth communications, it's time to start experimenting with the ESP32.

We will be using the Arduino IDE for our demonstrations. I assume that you already have your IDE installed and that you have also installed the ESP32 Boards Manager if necessary (the latest versions of the Arduino IDE 2.x have the ESP32 Boards Manager already installed).

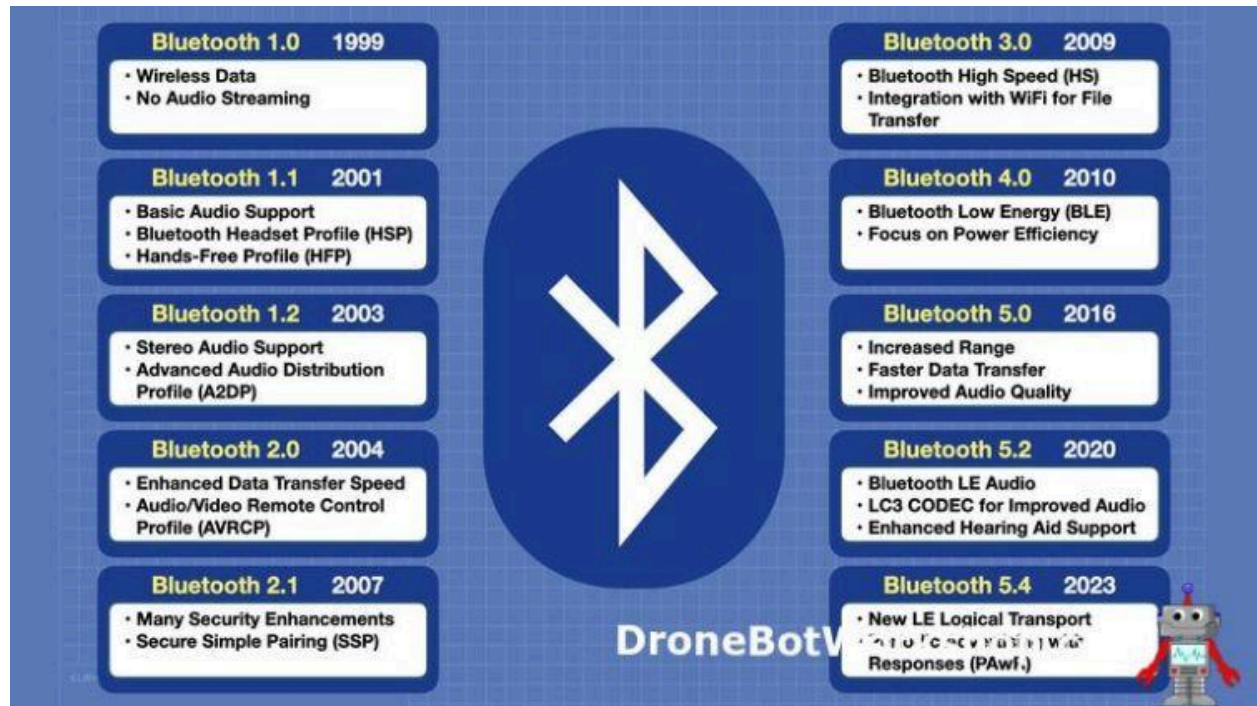
As for an ESP32, you could use any module that supports Bluetooth. You don't have to use the same module on both sides for experiments with servers and clients that require two ESP32 boards.

However, not every ESP32 supports Bluetooth; among those that do, the support for different Bluetooth versions differs.

Bluetooth Version History

Since the introduction of Bluetooth 1.0 in 1999, the protocol has undergone many changes. The most significant was the introduction of low-energy (BLE) in version 4.0.

The following chart illustrates some of the changes made with different versions of the Bluetooth standard. Note that there are other versions that are not shown on the chart.

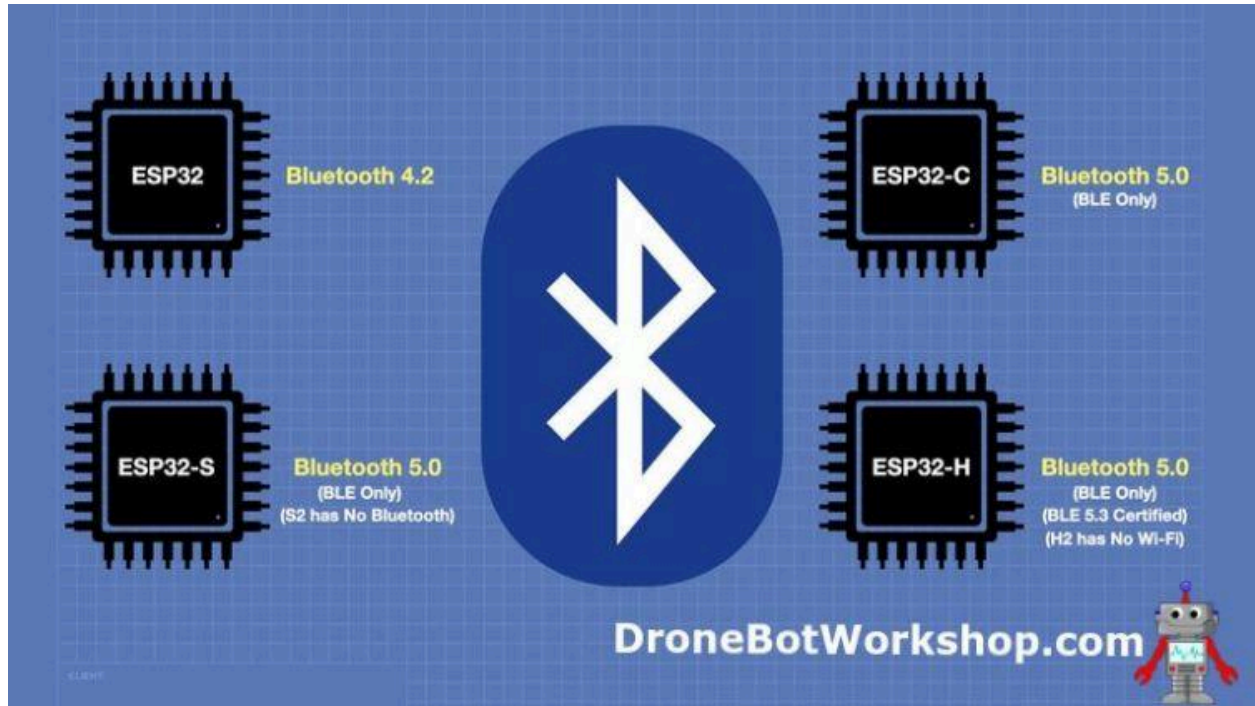


One recent Bluetooth improvement was in version 5.2, which was released in 2020. This version introduced Bluetooth LE Audio, a low-powered audio signal exchange method.

ESP32 Bluetooth Support

Support for Bluetooth versions differs between models of the ESP32, as shown on the following chart:

For more projects and tutorials visit the DroneBot Workshop - <https://dronebotworkshop.com>



All ESP32 boards with Bluetooth support also support BLE, as that was introduced in version 4. Some ESP32 boards support version 5.

Note that this information is current as of May 2024 and is subject to update.

<https://dronebotworkshop.com>

ESP32 & Bluetooth Classic

Finally, we are moving on to the experiments!

We will begin with Bluetooth Classic. The ESP32 has always supported Bluetooth Classic, and all the libraries and sample code you need to get started are included when you install the ESP32 Boards Manager in the Arduino IDE.

Our Bluetooth Classic experiments will require a board with an ESP32 chip—not an ESP32S or ESP32C chip. Bluetooth Serial needs BLE Classic (BR / EDR) with SPP over RFCOMM, which is only available in the regular ESP32.

Classic Bluetooth Profiles & Protocols

A *Bluetooth Stack* is software that implements the various layers of the Bluetooth protocol. Classic Bluetooth communications in the ESP32 are performed using the [BLUEDROID Bluetooth Stack](#), a popular open-source Bluetooth stack.

Two components define how data is formatted and used within the BLUEDROID stack:

- **Bluetooth Profiles** – These determine the overall function of each protocol layer.
- **Bluetooth Protocols** – These define the message formats and procedures.

The BLUEDROID stack in the ESP32 supports the following Profiles and Protocols:

Classic Bluetooth Profiles



The following Classic Bluetooth Profiles are supported:

GAP

GAP, or Generic Access Profile, is the most basic Bluetooth profile. *GAP* manages how devices discover and establish a connection with one another, as well as device security. *GAP* is used with both Bluetooth Classic and BLE.

A2DP (SNK) & AVRCP (CT)

The *A2DP* (Advanced Audio Distribution Profile) *SNK* (Sink), and *AVRCP* (Audio/Video Remote Control Profile) *CT* (Controller) profiles are used to stream high-quality audio.

For more projects and tutorials visit the DroneBot Workshop - <https://dronebotworkshop.com>

A2DP SNK manages how a device receives the audio stream from a source, such as a smartphone, MP3 player, or computer. It handles all of the audio stream.

AVRCP CT, on the other hand, manages the control aspect of the audio stream. This includes actions such as play, pause, stop, and volume control.

Classic Bluetooth Protocols



The following Classic Bluetooth Protocols are supported:

L2CAP

The *Logical Link Control and Adaptation Protocol*, or L2CAP, organizes Bluetooth traffic. It supports higher-level protocol multiplexing, quality of service, and data transmission.

<https://dronebotworkshop.com>

For more projects and tutorials visit the DroneBot Workshop - <https://dronebotworkshop.com>

L2CAP allows for the efficient transmission of large files and data, aside from audio streaming.

SDP

Service Discovery Protocol (SDP) enables Bluetooth devices to discover services (such as audio streaming or file transfer) provided by other devices.

AVDTP

Audio/Video Distribution Transport Protocol (AVDTP) supports the transmission of audio and video data over Bluetooth.

AVCTP

Audio/Video Control Transport Protocol (AVCTP) is used to transmit control commands between Bluetooth devices, such as remote controls and media players.

ESP32 Bluetooth Serial Library

Communications using Classic Bluetooth on the ESP32 use serial data. The Espressif [BluetoothSerial Library](#) simplifies working with the Bluetooth serial interface. It makes working with Classic Bluetooth almost as easy as working with a serial connection.

The library is installed in the Arduino IDE with the ESP32 Boards Manager, so you already have it. You can use it as follows:

1 – Include the Library:

<https://dronebotworkshop.com>

For more projects and tutorials visit the DroneBot Workshop - <https://dronebotworkshop.com>

Add this line at the top of your Arduino sketch:

```
1 #include <BluetoothSerial.h>
```

2 – Create a BluetoothSerial Object:

Define an object; we will call it *SerialBT* (you can give it another name if you wish):

```
1 BluetoothSerial SerialBT;
```

3 – Setup:

In your Setup function, start Bluetooth and provide a device name. This example uses *MyESP32BT* as the device name:

```
1 SerialBT.begin("MyESP32BT");
```

4 – Send and Receive Data:

Using Bluetooth Serial is similar to using Serial:

```
1 SerialBT.write() //Send data to the connected device.  
2  
3 SerialBT.available() //Check if there is data to read.  
4  
5 SerialBT.read() //Read incoming data from Bluetooth buffer.
```

To demonstrate the operation of the BluetoothSerial library, we will look at some example sketches.

Device Discovery – *bt_classic_device_discovery*

Sketch

For more projects and tutorials visit the DroneBot Workshop - <https://dronebotworkshop.com>

We will start with a simple example sketch that Espressif provides with the ESP32 Boards Manager. This is the *bt_classic_device_discovery* sketch, and it is found under *File/Examples/Examples for ESP32/BluetoothSerial* in the Arduino IDE.

The sketch performs two types of scans for Bluetooth Classic devices:

- **Asynchronous Scanning** – This is a Continuous and ongoing scan. It allows the device to discover other Bluetooth devices without blocking other tasks.
- **Synchronous Scanning** – A one-time operation that runs for a preset duration. Other tasks on the MCU are blocked while running a synchronous scan.

```
1 #include <BluetoothSerial.h>
2
3 #if !defined(CONFIG_BT_ENABLED) || !defined(CONFIG_BLUEDROID_ENABLED)
4 #error Bluetooth is not enabled! Please run `make menuconfig` to and enable it
5 #endif
6
7 #if !defined(CONFIG_BT_SPP_ENABLED)
8 #error Serial Bluetooth not available or not enabled. It is only available for the
   ESP32 chip.
9
10 #endif
11
12 BluetoothSerial SerialBT;
13
14 #define BT_DISCOVER_TIME 10000
15
16 static bool btScanAsync = true;
17 static bool btScanSync = true;
18
19 void btAdvertisedDeviceFound(BTAdvertisedDevice *pDevice) {
20     Serial.printf("Found a device asynchronously: %s\n",
21 pDevice->toString().c_str());
22 }
23
24 void setup() {
25     Serial.begin(115200);
26     SerialBT.begin("ESP32test"); //Bluetooth device name
27     Serial.println("The device started, now you can pair it with bluetooth!");
28
29     if (btScanAsync) {
```

For more projects and tutorials visit the DroneBot Workshop - <https://dronebotworkshop.com>

```
29   Serial.print("Starting asynchronous discovery... ");
30   if (SerialBT.discoverAsync(btAdvertisedDeviceFound)) {
31       Serial.println("Findings will be reported in \"btAdvertisedDeviceFound\"");
32       delay(10000);
33       Serial.print("Stopping discoverAsync... ");
34       SerialBT.discoverAsyncStop();
35       Serial.println("stopped");
36   } else {
37       Serial.println("Error on discoverAsync f.e. not working after a
38   \"connect\"");
39   }
40 }
41
42 if (btScanSync) {
43     Serial.println("Starting synchronous discovery... ");
44     BTScanResults *pResults = SerialBT.discover(BT_DISCOVER_TIME);
45     if (pResults) {
46         pResults->dump(&Serial);
47     } else {
48         Serial.println("Error on BT Scan, no result!");
49     }
50 }
51
52 void loop() {
53     delay(100);
54 }
```

The sketch starts by including the BluetoothSerial library, as described earlier.

For more projects and tutorials visit the DroneBot Workshop - <https://dronebotworkshop.com>

The next lines are precompiler statements; the compiler uses them to ensure that the hardware supports the Bluetooth Classic functions required by the sketch.

We then define a Serial Bluetooth object called *SerialBT*.

The *BT_DISCOVER_TIME* constant defined on the next line sets the time limit for synchronous scanning (in milliseconds, the default is 10 seconds).

We then define a couple of boolean status variables to represent synchronous and asynchronous modes.

The *btAdvertisedDeviceFound* function is a “callback function” that is called every time a new device is found during an asynchronous scan. It prints the results to the serial monitor.

In Setup, we start both the serial monitor and the Bluetooth Serial device. Once the Bluetooth serial device has started, we will print the status to the serial monitor.

Next we start an asynchronous scan. The following line starts the scan and sets the callback function, which we defined earlier:

```
1 SerialBT.discoverAsync(btAdvertisedDeviceFound)
```

The discovery runs for ten seconds. If a device is discovered during this time, its parameters are printed to the serial monitor using the callback function. After the time has elapsed, we exit the asynchronous scan.

After the asynchronous scan has finished we perform a single synchronous scan using the following statement:

```
1 SerialBT.discover(BT_DISCOVER_TIME)
```

After ten seconds (defined by *BT_DISCOVER_TIME*) we stop the scan and examine the buffer for data. If there is any, we print it to the serial monitor.

For more projects and tutorials visit the DroneBot Workshop - <https://dronebotworkshop.com>

All the code runs in the Setup, so no code (other than a short delay) is run in the Loop.

Running the Scan

Load the sketch onto your ESP32 board. It must be an ESP32, not an “S” or “C” variant, or it will fail the precompiler checks.

Now open the serial monitor and reset the board. Assuming you have any Bluetooth devices (try a phone or Smart TV), you will see them listed, along with their signal strength.

This is a simple example of how to scan, and you can use it as the basis for other code.

Serial Bluetooth Demo Sketch

For our next demonstration, we will communicate using Bluetooth Classic. We'll exchange text between the Serial Monitor in our Arduino IDE and a Terminal application on a phone or tablet.

Terminal App (Android or iOS)

To prepare for the next demonstration, you will require a Terminal application on your phone or tablet. There are several of these apps for Android and iOS devices.

I used the free [Serial Bluetooth Terminal by Kai Morich](#) for my experiments, which I installed on my Android phone via the Google Play Store. This is a very basic terminal app, but it is quite suitable for our ESP32 experiments.

<https://dronebotworkshop.com>

For more projects and tutorials visit the DroneBot Workshop - <https://dronebotworkshop.com>

Bluetooth Serial Client Sketch

The following sketch is very simple and is an adaptation of a standard Bluetooth Serial code sample. You can use it as the basis for your own Bluetooth data transfer sketch.


```
1  /*
2   Bluetooth Serial Client Demo
3   bluetooth_serial_client.ino
4   Demonstrates operation of BluetoothSerial Library
5   Based upon open-source code sample by Evandro Copercini - 2018
6
7   DroneBot Workshop 2024
8   https://dronebotworkshop.com
9  */
10
11 // Include BluetoothSerial library.
12 #include "BluetoothSerial.h"
13
14 // Name of Bluetooth client.
15 String device_name = "ESP32-BT-Client";
16
17 // Check if Bluetooth is available.
18 #if !defined(CONFIG_BT_ENABLED) || !defined(CONFIG_BLUEDROID_ENABLED)
19 #error Bluetooth is not enabled!
20 #endif
21
22 // Check if Serial Port Profile (SPP) is available.
23 #if !defined(CONFIG_BT_SPP_ENABLED)
24 #error Serial Port Profile for Bluetooth is not available or not enabled!
25 #endif
26
27 // Create a BluetoothSerial object.
28 BluetoothSerial SerialBT;
```

```
29
30 void setup() {
31   // Start Serial Monitor.
32   Serial.begin(115200);
33
34   // Start Bluetooth Serial.
35   SerialBT.begin(device_name);
36
37   // Uncomment the next line to delete previously paired devices. Must be called
   directly after bluetooth begin.
38   //SerialBT.deleteAllBondedDevices();
39
40   // Print to serial monitor.
41   Serial.printf("The device \"%s\" is started and can be paired with Bluetooth.\n",
42 device_name.c_str());
43 }
44
45 void loop() {
46   // Check for message from serial monitor.
47   if (Serial.available()) {
48     // Write message to paired Bluetooth device.
49     SerialBT.write(Serial.read());
50   }
51
52   // Check for message from paired Bluetooth device.
53   if (SerialBT.available()) {
54     // Write message to serial monitor.
55     Serial.write(SerialBT.read());
56   }
```

For more projects and tutorials visit the DroneBot Workshop - <https://dronebotworkshop.com>

```
57  delay(20);  
    }
```

We start by Including the *BluetoothSerial* library and defining a string representing our Bluetooth client name. You may change this string if you wish.

Once again, we use a couple of compiler checks to ensure we are running on Bluetooth-capable hardware. We then create a Serial Bluetooth object called *SerialBT*.

In Setup, we start both our serial monitor and or Bluetooth Serial object.

The following line is remarked out. It is used to delete any stored Bluetooth pairing information during setup. Uncomment it is you wish to do this during your experiments.

```
1 //SerialBT.deleteAllBondedDevices();
```

Finally, we print our device status on the serial monitor and exit the setup routine.

In the Loop, we check to see if anything has been typed into the serial monitor. If there is something there, we write it to the Bluetooth Serial object.

We then do the opposite, checking to see if there is any Bluetooth data available. If there is, we write it to the serial monitor.

After a very short delay, we do it all again.

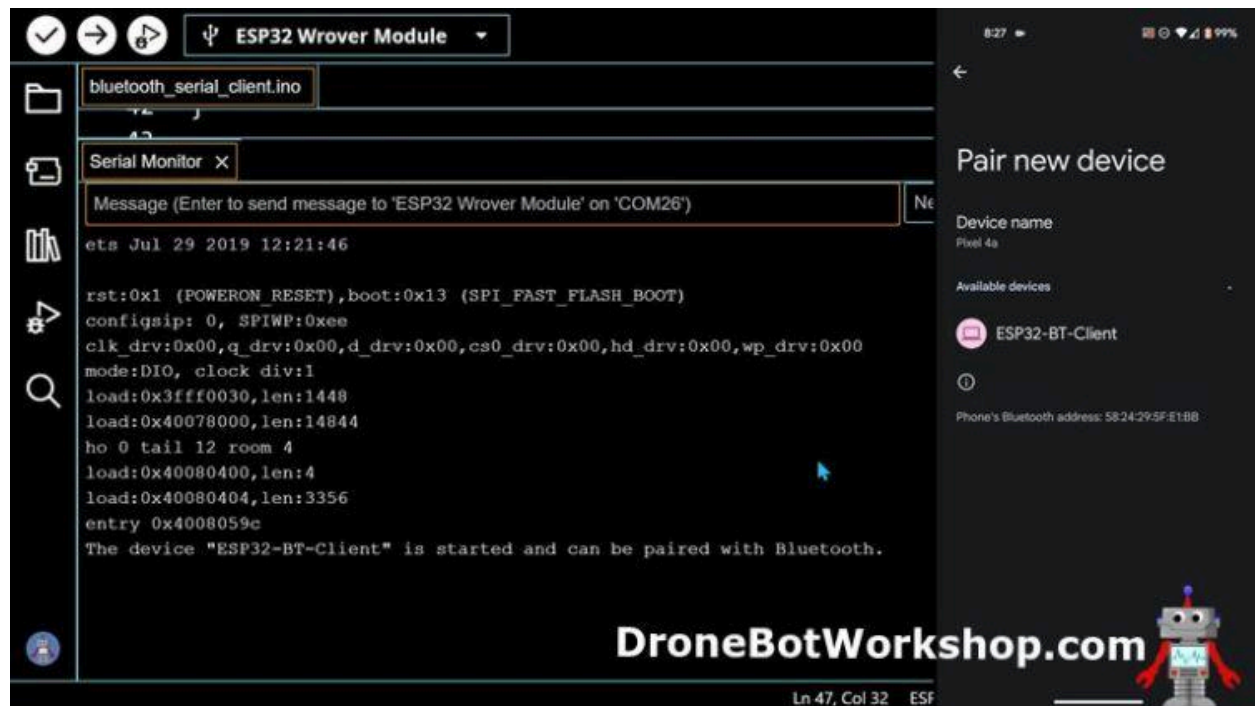
Running a Demo

You can demonstrate the sketch using your Arduino IDE Serial Monitor and the Terminal application that you installed on your phone or tablet.

For more projects and tutorials visit the DroneBot Workshop - <https://dronebotworkshop.com>

First, compile and load the sketch onto your ESP32 board. Open the serial monitor and reset the board. You should see the usual ESP32 startup information, followed by the device status line from the sketch.

Now go to your mobile device and open your Bluetooth control panel (the exact operation of this varies between phones, so the instructions here are generic). Scan for Bluetooth devices.



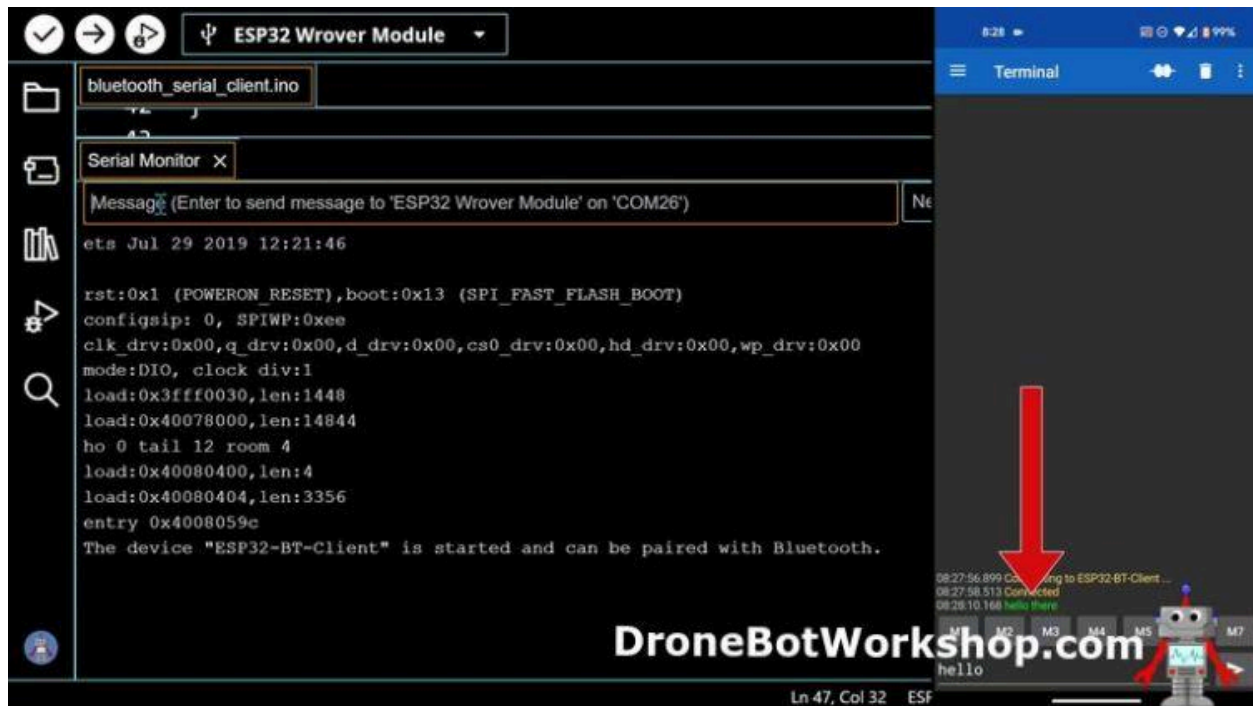
You should see the *ESP32-BT-Client* in your list of nearby devices. Select it and pair with it; no PIN number is required.

Now open the Terminal app. You will need to select the Bluetooth device to connect to. If you are using the same Serial Terminal app I am using, click on the menu icon at the top left corner and select Devices. You should see the *ESP32-BT-Client* device listed. Select it, and return to the Terminal selection.

<https://dronebotworkshop.com>

For more projects and tutorials visit the DroneBot Workshop - <https://dronebotworkshop.com>

Now, type something into the serial monitor. You should see the same line of text appear on your terminal app. After verifying that, type something into the Terminal app. It should appear on your serial monitor.



As this simple demonstration illustrates, serial Bluetooth makes it easy to send text or other data between Bluetooth devices.

<https://dronebotworkshop.com>

ESP32 & BLE

Now that we have seen how to work with Classic Bluetooth, we will focus our attention on Bluetooth Low Energy or BLE. This is an ideal protocol for battery-powered IoT or wearable devices.

Bluetooth Low Energy (BLE) Profiles

As with Bluetooth Classic, we use Profiles to establish our connection and exchange data with BLE. Two profiles are commonly used with BLE: GAP and GATT.

GAP (Generic Access Profile)

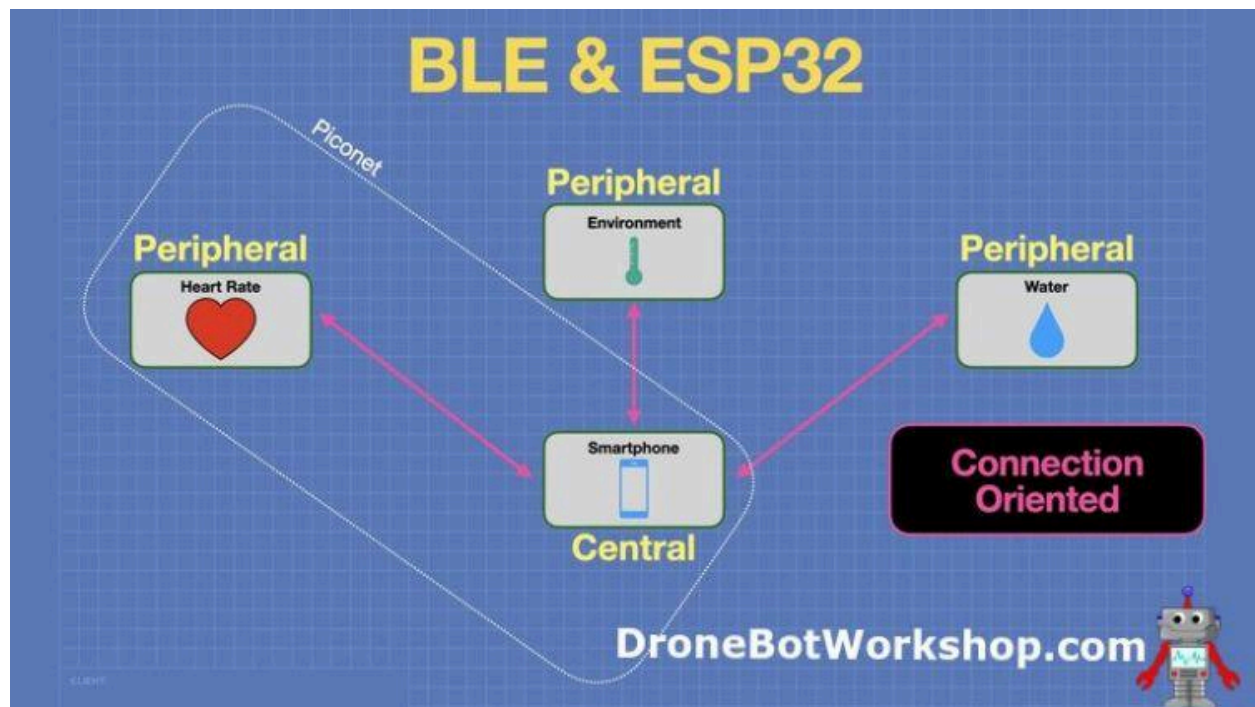
We have already seen the Generic Access Profile used the Bluetooth Classic. It is also used with BLE.

GAP handles all of the BLE device discovery, connection, security, and advertising functions. It is essential when establishing a connection between BLE devices.

GAP also determines the network topology, as there are a couple of different topologies used with BLE.

Connection-Oriented

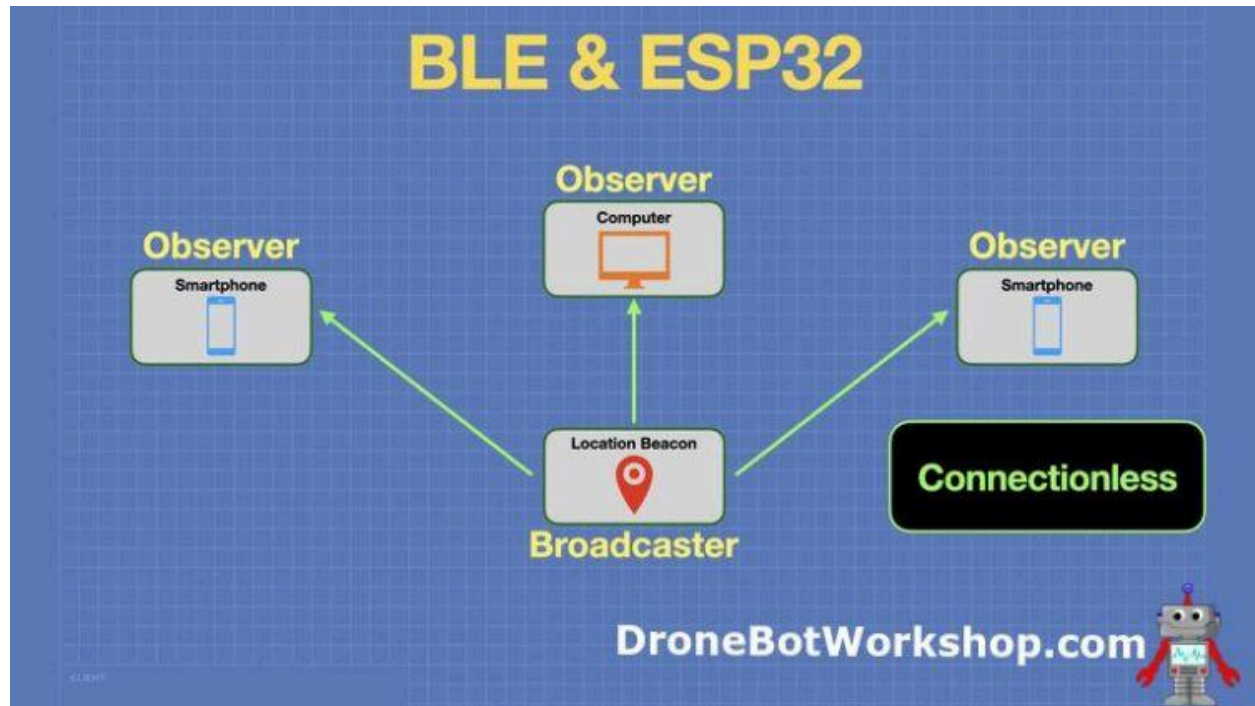
In a connection-oriented topology, a Central device, such as a smartphone or computer, connects to one or more BLE Peripherals. The Central device and its Peripherals communicate bi-directionally.



A connection needs to be established between each Peripheral and the Central device, this connection is often referred to as a Piconet.

Connectionless

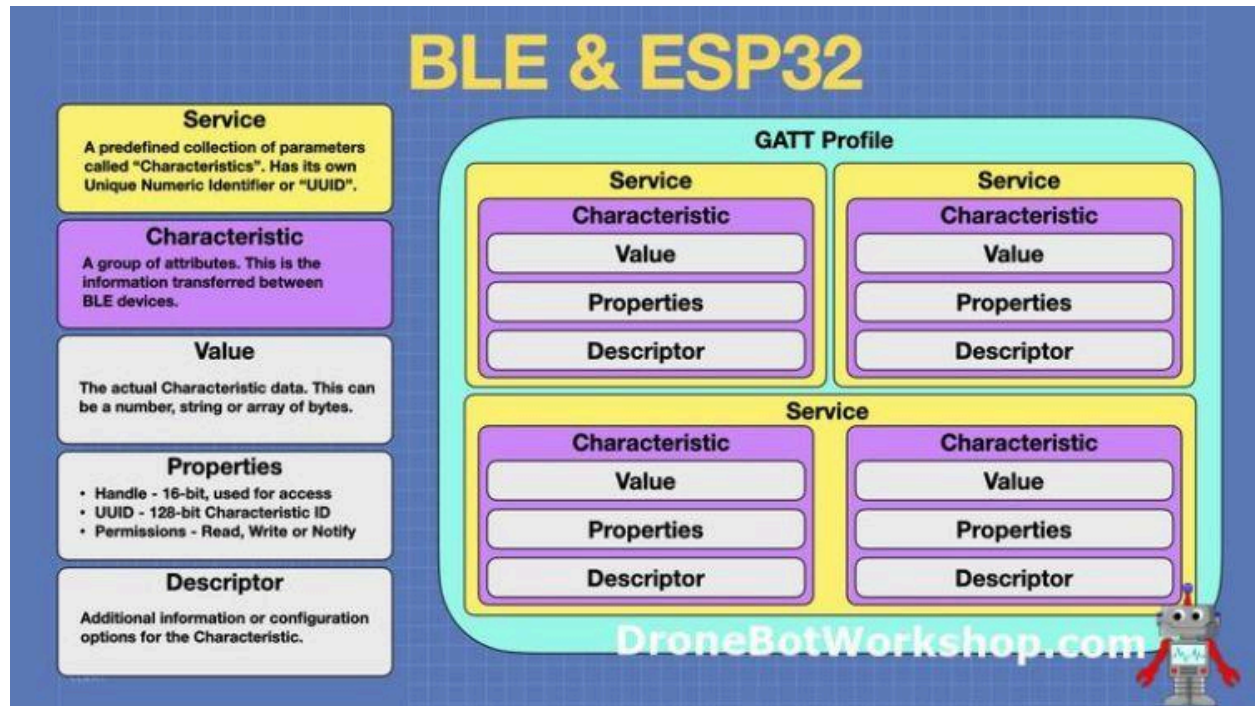
In a connectionless topology, the BLE device acts like a broadcaster. Devices like computers, tablets, and smartphones receive the broadcast periodically.



The broadcaster does not permit connections; observers simply scan for data packets. Communication between the broadcaster and the observer(s) is unidirectional. The broadcaster is incapable of receiving data from the observers and is unaware of how many observer devices there are.

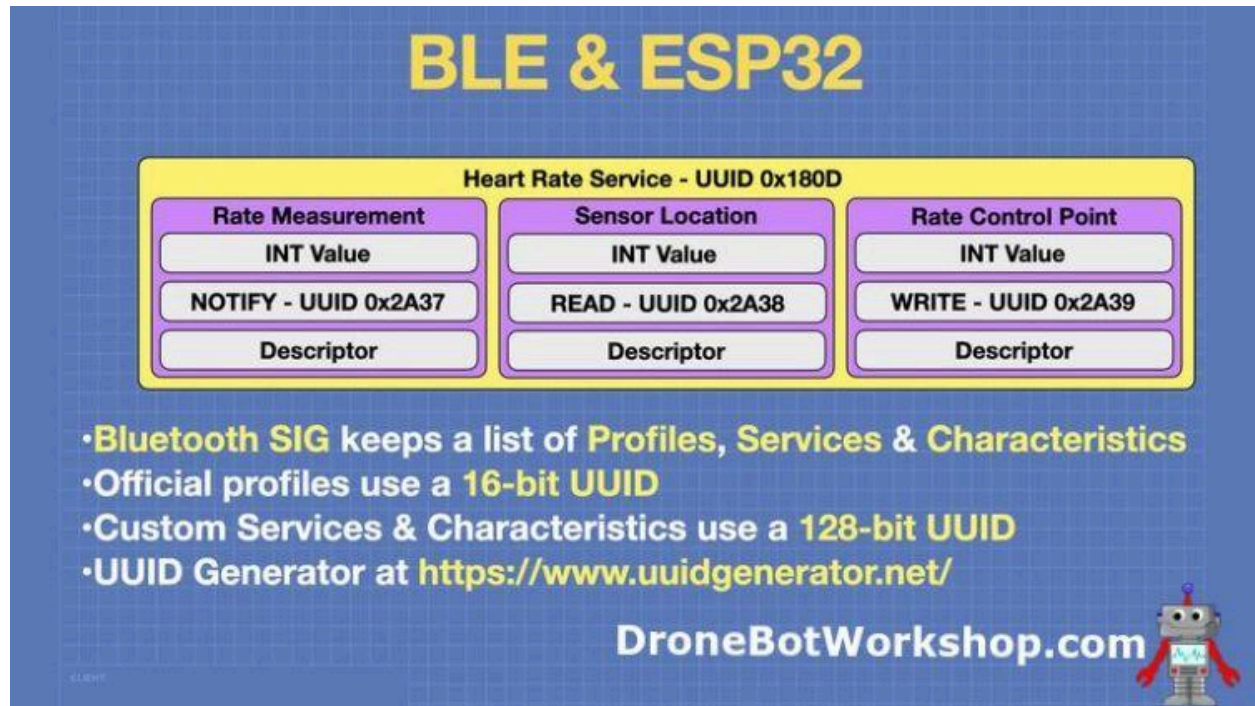
GATT (Generic ATtribute Profile)

The Generic Attribute Profile, or GATT, determines the format of the data exchanged between BLE devices. Data is sent in a format of attributes and values. It is built upon a more generic profile called ATT for the Attribute Profile.



Services

A GATT Service is a collection of parameters called Characteristics. Each service can have one or more Characteristics.



An example of a GATT Service is the Heart Rate Service. It has a 16-bit UUID of 0x180D and contains three characteristics:

- Heart Rate Measurement
- Body Sensor Location
- Heart Rate Control Point.

This is an officially defined service. The Bluetooth SIG (Special Interest Group) maintains a list of officially defined Services at <https://www.bluetooth.com/specifications/gatt/services>.

Characteristics

As mentioned above, a Service is a collection of parameters called Characteristics. The Characteristics are the actual data transferred between Bluetooth devices.

For more projects and tutorials visit the DroneBot Workshop - <https://dronebotworkshop.com>

Every Characteristic has its own UUID, and once again, the Bluetooth SIG keeps a list of official Characteristics.

A Service can have more than one Characteristic. The Heart Rate Service we looked at earlier has three Characteristics:

- Rate Measurement – UUID 0x2A37
- Sensor Location – UUID 0x2A38
- Rate Control Point – UUID 0x2A39

You may also define your own Characteristics for a custom Bluetooth device. Custom Characteristics will use a 128-bit UUID, which you can generate [using this website](#).

Characteristics have three sections: Values, Properties, and Descriptors.

Values

Values are the actual data contained in a Characteristic. For example, in the Rate Measurement Characteristic that we described earlier the Value would be a numeric representation of the heart rate in beats per minute.

Values can be numbers, strings, or an array of bytes.

Properties

The Properties section has three values:

- Handle – A 16-bit identifier that is used by the server to address the Characteristic.
- UUID – This is where the Characteristic UUID is inserted.
- Permissions – The permissions assigned to the Characteristic.

Descriptors

The Descriptors section is for additional information or configuration settings. Not every Characteristic will have a Descriptor.

ESP32 BLE Libraries

Now that we understand how Bluetooth Low Energy operates, we can turn our attention to the libraries provided by Espressif for using BLE with the ESP32.



The ESP32 can act as either a BLE Server or a Client. There are several libraries provided to support both configurations.

- **BLEDevice** – This library provides functions to initialize and configure the BLE stack on the ESP32. It allows you to create a BLE server or client, set up services and characteristics, and handle BLE events.

- **BLEServer** – This library allows you to create a BLE server on the ESP32. You can add services, characteristics, and descriptors to the server.
- **BLEClient** – This library lets you create a BLE client on the ESP32. It lets you scan for and connect to BLE peripherals, read and write characteristics, and handle notifications.
- **BLEUtils** – The BLEUtils library provides utility functions for working with BLE on the ESP32. It includes functions for converting data types, managing UUIDs, and handling BLE advertising.
- **BLEScan** – This library allows you to perform BLE scanning on the ESP32 to discover nearby BLE devices and retrieve information about them.
- **BLEAdvertisedDevice** – This library retrieves information about the scanned device, such as its address and the services it provides.

In addition Espressif provides several example sketches for working with BLE. We will run a few of them now.

Sample Sketch – Server

The *Server* sketch can be found at *File/Examples/Examples for ESP32/BLE/*.

This sketch illustrates the operation of a simple BLE Server. As a server, it does the following:

- It **Advertises** itself as a Bluetooth device, making it visible to phones, tablets, computers, and other ESP32s.
- Offers a **Service** with unique **Characteristics**.
- It permits other devices to read and write data to these Characteristics.

Here is the code:

For more projects and tutorials visit the DroneBot Workshop - <https://dronebotworkshop.com>

```
1 /*
2     Based on Neil Kolban example for IDF:
3     https://github.com/nkolban/esp32-snippets/blob/master/cpp_utils/tests/BLE%20Tests/Sa
4     mpleServer.cpp
5     Ported to Arduino ESP32 by Evandro Copercini
6     updates by chegewara
7 */
8 #include <BLEDevice.h>
9 #include <BLEUtils.h>
10 #include <BLEServer.h>
11
12 // See the following for generating UUIDs:
13 // https://www.uuidgenerator.net/
14
15 #define SERVICE_UUID          "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
16 #define CHARACTERISTIC_UUID   "beb5483e-36e1-4688-b7f5-ea07361b26a8"
17
18 void setup() {
19     Serial.begin(115200);
20     Serial.println("Starting BLE work!");
21
22     BLEDevice::init("Long name works now");
23     BLEServer *pServer = BLEDevice::createServer();
24     BLEService *pService = pServer->createService(SERVICE_UUID);
25     BLECharacteristic *pCharacteristic =
26         pService->createCharacteristic(CHARACTERISTIC_UUID,
27         BLECharacteristic::PROPERTY_READ | BLECharacteristic::PROPERTY_WRITE);
28
29     pCharacteristic->setValue("Hello World says Neil");
30 }
```

<https://dronebotworkshop.com>


```
2 pService->start();
2 // BLEAdvertising *pAdvertising = pServer->getAdvertising(); // this still is
2 working for backward compatibility
3 BLEAdvertising *pAdvertising = BLEDevice::getAdvertising();
2 pAdvertising->addServiceUUID(SERVICE_UUID);
4 pAdvertising->setScanResponse(true);
2 pAdvertising->setMinPreferred(0x06); // functions that help with iPhone
5 connections issue
2 pAdvertising->setMinPreferred(0x12);
6 BLEDevice::startAdvertising();
2 Serial.println("Characteristic defined! Now you can read it in your phone!");
7 }
2
9 void loop() {
3 // put your main code here, to run repeatedly:
0 delay(2000);
3
1 }
```

4
0
4
1
4
2

The first thing we do is include three BLE libraries:

- **BLEDevice** – Core BLE function library.
- **BLEUtils** – Utilities for working with BLE.
- **BLEServer** – BLE Server library.

Next we define constants for the UUID of both the *Service* and its *Characteristic*.

Remember that you can generate your own UUIDs using the online UUID Generator.

We will just use the ones in the example for our demonstration.

In Setup, we start the serial monitor and print to it.

We then initialize the BLE stack and provide a name for the device.

After we do that, we create a BLEServer variable named *pService*. We then use *pService* to create a *Service* and a *Characteristic* for that service. Our Service has a UUID, and our Characteristic has read and write properties and its own UUID.

We then set the value of our Characteristic to “*Hello World says Neil*”. If your name is not Neil, you can edit this!

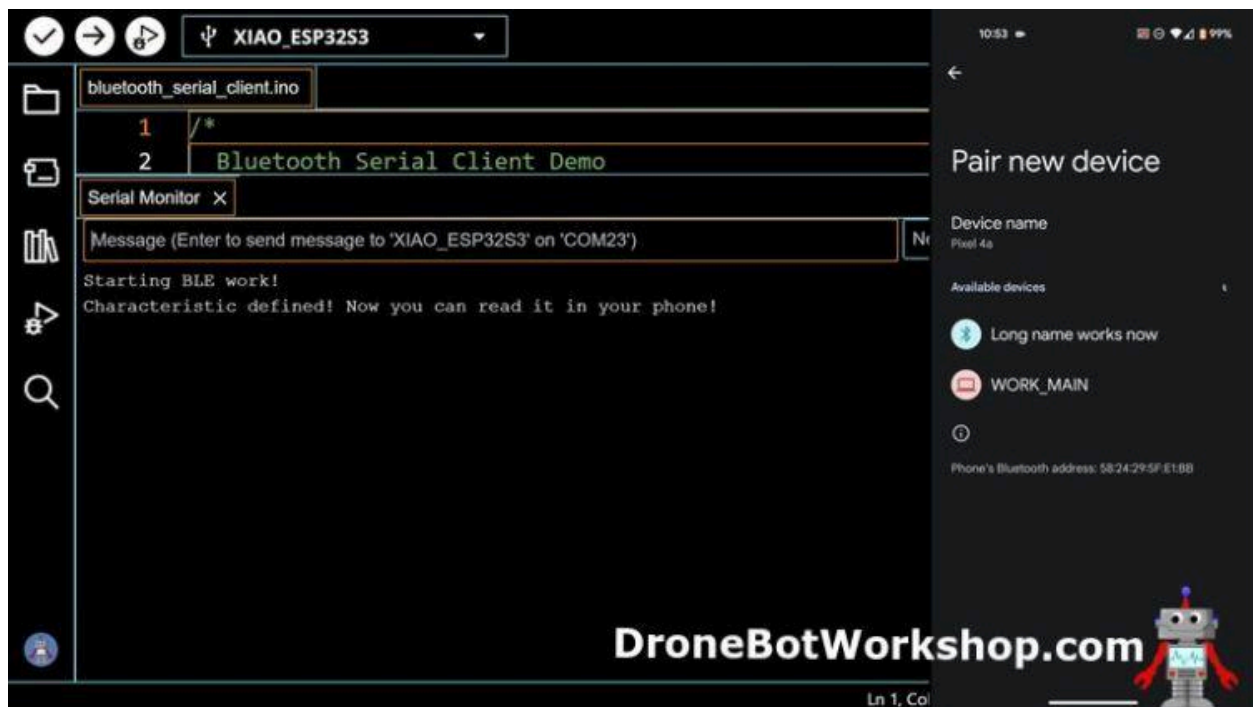
Finally, we start the Service.

After starting the service, we can advertise it to other BLE-enabled devices. We do this with a BLE Advertising object named *pAdvertising*. We then add some parameters to *pAdvertising* and then start BLE advertising.

Testing the BLE Server

There are a few different methods of testing the BLE Server.

The simplest method is to see if you can see it in the list of Bluetooth devices available to connect to your phone or tablet. It should have that ridiculous name “Long name works now”!



You can then use an app like [Nordic nRF Connect](#) to attach to the Service and modify the data in the Characteristic.

Another method is just to use another ESP32 as a BLE Client and attach it to the server. Let's see how we would do that.

Sample Sketch – Client

For more projects and tutorials visit the DroneBot Workshop - <https://dronebotworkshop.com>

A corresponding sample sketch is the BLE Client sketch, located in the same folder as the Server sketch. The sketch was written to work with the BLE Server we just built.

The BLE Client performs the following operations:

- **Scans** for nearby BLE Servers (BLE devices that offer data).
- Connect to a specific **Service** if it finds a matching UUID.
- Reads and writes to **Characteristics** in that Service.

Here is the Client sketch:

```
1 /**
2  * A BLE client example that is rich in capabilities.
3  * There is a lot new capabilities implemented.
4  * author unknown
5  * updated by chegewara
6  */
7
8 #include "BLEDevice.h"
9 // #include "BLEScan.h"
10
11 // The remote service we wish to connect to.
12 static BLEUUID serviceUUID("4fafc201-1fb5-459e-8fcc-c5c9c331914b");
13 // The characteristic of the remote service we are interested in.
14 static BLEUUID charUUID("beb5483e-36e1-4688-b7f5-ea07361b26a8");
15
16 static boolean doConnect = false;
17 static boolean connected = false;
18 static boolean doScan = false;
19 static BLERemoteCharacteristic *pRemoteCharacteristic;
20 static BLEAdvertisedDevice *myDevice;
21
22 static void notifyCallback(BLERemoteCharacteristic *pBLERemoteCharacteristic,
23   uint8_t *pData, size_t length, bool isNotify) {
24   Serial.print("Notify callback for characteristic ");
25   Serial.print(pBLERemoteCharacteristic->getUUID().toString().c_str());
26   Serial.print(" of data length ");
27   Serial.println(length);
28   Serial.print("data: ");
29   Serial.write(pData, length);
```

```
29 Serial.println();
30 }
31
32 class MyClientCallback : public BLEClientCallbacks {
33     void onConnect(BLEClient *pclient) {}
34
35     void onDisconnect(BLEClient *pclient) {
36         connected = false;
37         Serial.println("onDisconnect");
38     }
39 };
40
41 bool connectToServer() {
42     Serial.print("Forming a connection to ");
43     Serial.println(myDevice->getAddress().toString().c_str());
44
45     BLEClient *pClient = BLEDevice::createClient();
46     Serial.println(" - Created client");
47
48     pClient->setClientCallbacks(new MyClientCallback());
49
50     // Connect to the remote BLE Server.
51     pClient->connect(myDevice); // if you pass BLEAdvertisedDevice instead of
    address, it will be recognized type of peer device address (public or private)
52     Serial.println(" - Connected to server");
53     pClient->setMTU(517); //set client to request maximum MTU from server (default is
54 23 otherwise)
55
56     // Obtain a reference to the service we are after in the remote BLE server.
```

```
57 BLERemoteService *pRemoteService = pClient->getService(serviceUUID);
58 if (pRemoteService == nullptr) {
59     Serial.print("Failed to find our service UUID: ");
60     Serial.println(serviceUUID.toString().c_str());
61     pClient->disconnect();
62     return false;
63 }
64 Serial.println(" - Found our service");
65
66 // Obtain a reference to the characteristic in the service of the remote BLE
server.
67 pRemoteCharacteristic = pRemoteService->getCharacteristic(charUUID);
68 if (pRemoteCharacteristic == nullptr) {
69     Serial.print("Failed to find our characteristic UUID: ");
70     Serial.println(charUUID.toString().c_str());
71     pClient->disconnect();
72     return false;
73 }
74 Serial.println(" - Found our characteristic");
75
76
77 // Read the value of the characteristic.
78 if (pRemoteCharacteristic->canRead()) {
79     String value = pRemoteCharacteristic->readValue();
80     Serial.print("The characteristic value was: ");
81     Serial.println(value.c_str());
82 }
83
84 if (pRemoteCharacteristic->canNotify()) {
    pRemoteCharacteristic->registerForNotify(notifyCallback);
```



```
85 }
86
87 connected = true;
88 return true;
89 }
90 /**
91  * Scan for BLE servers and find the first one that advertises the service we are
92  * looking for.
93  */
94 class MyAdvertisedDeviceCallbacks : public BLEAdvertisedDeviceCallbacks {
95     /**
96      * Called for each advertising BLE server.
97      */
98     void onResult(BLEAdvertisedDevice advertisedDevice) {
99         Serial.print("BLE Advertised Device found: ");
100         Serial.println(advertisedDevice.toString().c_str());
101
102         // We have found a device, let us now see if it contains the service we are
103         // looking for.
104         if (advertisedDevice.haveServiceUUID() &&
105             advertisedDevice.isAdvertisingService(serviceUUID)) {
106
107             BLEDevice::getScan()->stop();
108
109             myDevice = new BLEAdvertisedDevice(advertisedDevice);
110
111             doConnect = true;
112
113             doScan = true;
114
115             } // Found our server
116     } // onResult
117 }; // MyAdvertisedDeviceCallbacks
```

```
10
8
void setup() {
10
9   Serial.begin(115200);
11
11   Serial.println("Starting Arduino BLE Client application...");
0
11   BLEDevice::init("");
1
11   // Retrieve a Scanner and set the callback we want to use to be informed when we
2   // have detected a new device. Specify that we want active scanning and start the
11   // scan to run for 5 seconds.
3
11   BLEScan *pBLEScan = BLEDevice::getScan();
4
11   pBLEScan->setAdvertisedDeviceCallbacks(new MyAdvertisedDeviceCallbacks());
11   pBLEScan->setInterval(1349);
5
11   pBLEScan->setWindow(449);
6
11   pBLEScan->setActiveScan(true);
11   pBLEScan->start(5, false);
7 } // End of setup.
11
8
// This is the Arduino main loop function.
11
9 void loop() {
12
0   // If the flag "doConnect" is true then we have scanned for and found the desired
12   // BLE Server with which we wish to connect. Now we connect to it. Once we are
1   // connected we set the connected flag to be true.
12
2   if (doConnect == true) {
12
12     if (connectToServer()) {
3
12       Serial.println("We are now connected to the BLE Server.");
4
12     } else {
12
12       Serial.println("We have failed to connect to the server; there is nothing more
5 we will do.");
```

```
12     }
6
12     doConnect = false;
12
7     }
12
12
8     // If we are connected to a peer BLE Server, update the characteristic each time
12 we are reached
9
12     // with the current time since boot.
13
13     if (connected) {
0
12         String newValue = "Time since boot: " + String(millis() / 1000);
13
1     Serial.println("Setting new characteristic value to \"" + newValue + "\"");
13
12
2         // Set the characteristic's value to be the array of bytes that is actually a
13 string.
3
12         pRemoteCharacteristic->writeValue(newValue.c_str(), newValue.length());
13
4     } else if (doScan) {
13
12         BLEDevice::getScan()->start(0); // this is just example to start scan after
5 disconnect, most likely there is better way to do it in arduino
13
12     }
6
13
12
7     delay(1000); // Delay a second between loops.
13
12 } // End of loop
```

```
14
4
14
5
14
6
14
7
14
8
14
9
15
0
15
1
15
2
15
3
15
4
15
5
15
6
```

The sketch starts by loading the *BLEDevice* library. A previous version also loaded the *BLEScan* library, but this is no longer required.

Next, we define the UUIDs of the remote Service and Characteristic we want to work with. This needs to match the values in the Server sketch, so if you changed the UUID, you will need to edit it here as well. For our purposes, we will just leave it alone.

For more projects and tutorials visit the DroneBot Workshop - <https://dronebotworkshop.com>

Next, we define a few global variables to keep track of the connection state. We also define two callbacks:

- **notifyCallback** – A function that is called automatically when the server sends a notification message about a changing value in a Characteristic. It prints the data to the serial monitor.
- **MyClientCallback** – A custom class defining the actions to take when a client connects or disconnects. In this case, it updates a “connected” flag when the client disconnects.

Our next entry is a boolean function called *connectToServer()*. It performs the following operations:

- Creates a BLE Client
- Connects to the remote BLE server
- Retrieves the Services and Characteristics
- Reads the Characteristics value
- Registers for notifications (if available)

After that is the *MyAdvertisedDeviceCallbacks* callback class; this class handles the discovered BLE devices. In particular, it checks the UUID of the remote device Service to see if it matches the one it is looking for. If it finds the Service, it sets the *doConnect* flag to TRUE (HIGH) to initiate a connection.

Now we have made it to the Setup.

We start the serial monitor, print to it, and then initiate the BLE Device. We don't need a name for the device as it is a client. We then set up a BLE scan to look for our desired Service UUID. We then exit Setup, with the scan running in the background.

In the Loop, we check the *doConnect* flag. If it is TRUE, we call the *connectToServer* function, the boolean function described earlier. If it comes back TRUE, we are indeed connected to the desired BLE server. We display a message and reset the flag.

<https://dronebotworkshop.com>

For more projects and tutorials visit the DroneBot Workshop - <https://dronebotworkshop.com>

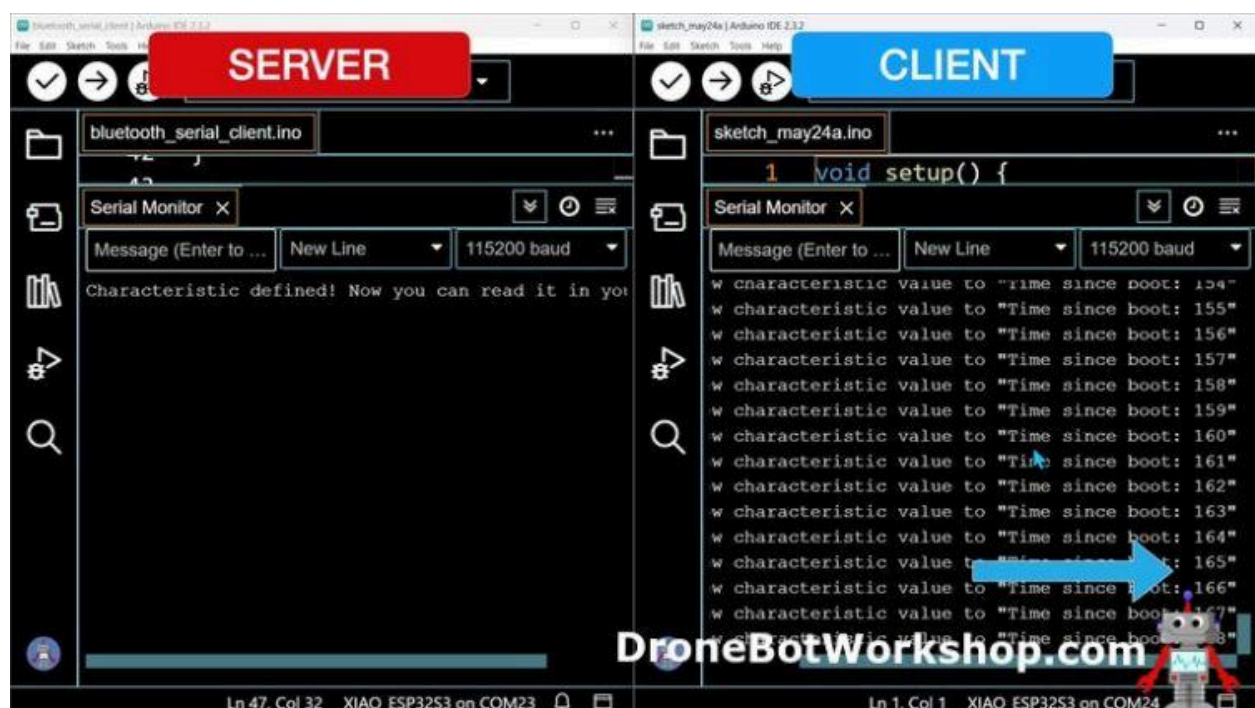
If we are connected to the remote server, we update the remote Characteristic with a timestamp value. The timestamp uses the *millis* function to measure the number of milliseconds since the ESP32 was restarted.

We then delay a second and return to the Loop's start.

Testing the BLE Client

We can test the BLE Client using the BLE Server we created earlier. The client is set to look for the Service and Characteristic UUIDs used on the Server.

You can run both the BLE Server and BLE Client ESP32 boards on the same computer with two instances of the Arduino IDE or on two different computers. Either way, start up each board, the Server before the Client.



For more projects and tutorials visit the DroneBot Workshop - <https://dronebotworkshop.com>

You should see the server start and print a message saying you can now attach your phone. On the client side you should see the value of the Characteristic being modified each second.

It's a simple demo, but you can use it to build BLE servers and Clients for your projects.

Conclusion

Using the ESP32 with Bluetooth Classic and BLE is easy once you understand how it functions and how to use the ESP32 libraries provided by Espressif.

You can use Bluetooth Classic to connect to existing Bluetooth devices or to build projects like custom Bluetooth Speakers. BLE is ideal for IoT projects and remote controls. Exchanging data between ESP32 devices via Bluetooth also opens up many possibilities for new and exciting projects.

King Harald Bluetooth would be pretty impressed with what his namesake can do!